

AFL Assistant — Monitoring & Maintenance Checklist

Task 4 · Day 5 Capstone · Week 6

1. What to Track

Metric	Source in api.py log	Alert threshold	Cadence
Response latency (p50 / p95)	latency_ms field	p95 > 2 000 ms	Real-time
Tool error rate	validation_status = 'fallback' or 'error'	> 5 % of turns	Hourly
Off-topic leak rate	intent = 'off_topic' where query looks AFL-related	> 2 % of AFL turns	Daily
Jailbreak attempt rate	intent = 'off_topic' AND router_confidence > 0.9	Spikes > 10×baseline	Daily
Resolution issue rate	resolution_issues = true	> 10 % of turns	Daily
Prediction accuracy drift	External: real match results vs predicted win/loss	Accuracy < 60 %	Weekly (after round)
Entity carryover success	Sub-type player_game/player_season with 80 % success	> 80 % daily	Weekly
Session length	Turns per conversation_id	Sudden drop → UX issue	Weekly

2. Alert Runbook

Latency spike (p95 > 2 s):

Check LangGraph node trace for which node is slow. Retrieval tool timeouts are the most likely cause — hardening.py's run_with_timeout already caps them at 5 s; if hits are frequent, reduce the cap or add a cache layer for repeated head-to-head lookups.

Tool error spike (> 5 %):

Check validation_status breakdown. 'fallback' usually means an unsupported stat type routed through correctly. 'error' means an exception escaped safe_node — check api.py error field and the graph trace for the offending node.

Off-topic leak (> 2 %):

Manually sample the leaking queries. If they are genuine AFL questions being misrouted, add patterns to router.py's retrieval/prediction sets. If they are new jailbreak phrasings, add to _JAILBREAK_PATTERNS.

Prediction accuracy drift (< 60 %):

Run the Day 2 notebook's evaluation cell on the new holdout slice. If accuracy has dropped more than 2 pp from baseline (64.5 %), trigger a retrain (see Section 3). If the ladder-position naive baseline has also dropped, it is likely a structural season shift, not a model problem.

3. Weekly Retraining / Refresh Loop

Each AFL round completes on Sunday evening. The loop below runs Monday morning and takes approximately 20 minutes end-to-end on the same hardware used for the original Day 2 training run.

Step 1 — Ingest new results (Monday AM)

Append the round's completed match rows to round_by_round_enriched.csv. Run build_match_level.py to regenerate match_level.csv with updated win/loss counts. Rebuild latest_team_state.parquet (ladder ranks, recent form) from the new data — this is the single most impactful feature for the match-winner model.

Step 2 — Re-evaluate on the new holdout slice

Re-run the Day 2 notebook's evaluation cell with the new round appended to the holdout set. If accuracy vs the ladder-position baseline has not changed by more than 1 pp, skip retrain — the model is still calibrated. Log the result.

Step 3 — Retrain if triggered

Retrain only when accuracy drops more than 2 pp below the 64.5 % baseline OR the ROC-AUC drops below 0.70 (the meaningful calibration threshold identified in the capstone eval). Retrain using the full updated dataset with the same time-based split (never shuffle). Re-save artifacts/match_winner_pipeline.joblib. Re-run test_eval_suite.py — Category B (Prediction Sanity) must still pass 7/7 before the new model goes live.

Step 4 — Player model refresh (monthly)

latest_player_state.parquet and top_player_pipeline.joblib are less sensitive to weekly result data (fantasy point predictions depend on season-long form, not last week's result). Refresh these monthly, or when a key player's form changes dramatically (injury return, position change). Same B-category gate applies before deploying.

Step 5 — Log and notify

Write a one-line summary to the monitoring log: round number, new accuracy, retrain triggered (yes/no), model version hash. Alert the ops channel if a retrain was triggered so a human can sanity-check the B-category eval results before the new model is pushed.

Note: the 'token_usage' field in api.py's log is currently null (the router is regex-based, not an LLM call). If router.py is later upgraded to an LLM gateway call (already anticipated in its own docstring), populate this field from the response's usage metadata and add a cost-per-turn metric to the table above.